Automatic bounds discovery for quantum error-budgeting: empirical validation of Deltakit issue #216

Everton Melo

forense.melo@protonmail.com

Abstract

Deltakit issue #216 asks for optional bounds on *get_error_budget* and an automatic explorer *find_bounds_auto* that grows an interval in log space until Lambda varies enough and the logical error rate stays in a feasible Monte Carlo range. This paper reports a validation suite on a repetition-code detector error model (distances 3, 5, 7) and a live call to the public Deltakit 0.9.0 API. Ten testbed hypotheses plus two API checks all pass. Common random numbers cut Var(ΔΛ) by 6.8× (ratio 0.147). A Pauli score-function gradient matches coupled finite differences at 10× fewer shots. Likelihood reweighting covers seven sweep points from one campaign. Log-space fits beat linear fits (R^2 0.992 vs 0.872). σΛ varies 19× inside a bracket; weighted least squares cuts derivative variance by 69.5%. c-optimal design beats Chebyshev nodes by 4.34×. A validity guard finds points where SNR is high but the Lambda model has already broken ($R^2 < 0.95$). Bootstrap σΛ is 1.37× the delta-method value. On Deltakit 0.9.0, auto-explored bounds [0.00125, 0.01] contain both the operating point p = 5×10−3 and the half-point p/2 at which the API evaluates the gradient; the auto contribution agrees with the documented baseline to 1.12σ. The return object still omits bounds and diagnostics (PO6).

Keywords: quantum error correction; error budgeting; Deltakit; common random numbers; score function; c-optimal design; automatic bounds.

1. Introduction

Error budgeting decomposes a logical-error scaling factor Λ into contributions from each physical noise parameter. The public Deltakit API does this by fitting 1/Λ on a user-supplied interval per parameter and taking a derivative at the half-point of the operating vector [1, 2]. Issue #216 [3] notes that those intervals are currently manual. A too-narrow interval drowns the signal in sampling noise. A too-wide interval leaves the asymptotic model $p_L \propto \Lambda^{-(d+1)/2}$ and produces a fit that no longer means Λ.

The issue asks for optional *bounds* on *get_error_budget* and a method *find_bounds_auto* that grows ε exponentially in log space until two conditions hold: Λ varies enough, and p_L stays inside a feasible range. It also flags Chebyshev nodes as a non-trivial choice for where to sample inside the bracket.

This paper does not replace the Deltakit implementation. It checks the hypotheses that would justify that implementation, on a controllable repetition-code testbed, and then calls the real API once those hypotheses pass. The testbed uses stim [4] and PyMatching [5] at the detector error model (DEM) level. Section 2 describes the suite. Section 3 reports numbers. Section 4 reads the results against the issue text. Section 5 lists what the API still does not return.

2. Methods

2.1 Testbed

A repetition-code memory experiment is compiled at distances $d \in \{3, 5, 7\}$ with a three-component noise vector p = (p_boundary, p_bulk, p_dummy0) at the operating point p* = 0.015. Lambda is obtained from a log-linear fit of logical error per round versus distance. Two sampling budgets are used. QUICK=True: 20 000 shots, 12 replicas, about 15 minutes. QUICK=False: 80 000 shots, 30 replicas, about 50 minutes. All testbed numbers below are QUICK=False unless stated. Global seed 20260216. Python 3.14.4 on WSL Ubuntu-26.04, with numpy 2.5.2, scipy 1.18.1, stim 1.16.0, pymatching 2.4.0.

2.2 Hypotheses

T1 (H2). Common random numbers (CRN): the same stim seed at both ends of a coupled pair. Pass if Var_CRN / Var_IND < 0.2.

T2 (H4). Pauli score-function gradient versus coupled finite differences. Pass if |z| < 2 and the score estimator uses fewer shots.

T3 (H3). Likelihood reweighting from a single campaign at p*, with effective sample size (ESS) as the stop statistic. Pass if at least three sweep points have relative error below 15%.

T4 (H7). Linear versus log parametrization of p. Pass if $R^2\_log \geq R^2\_lin$.

T5 (H6). Heteroscedasticity of σΛ inside a bracket; weighted least squares (WLS) versus ordinary least squares (OLS). Pass if σΛ spreads by at least 3× and WLS variance is lower.

T6 (H5, PO4). c-optimal design for the derivative (Elfving 1952; Pukelsheim 2006) versus Chebyshev nodes and exponential reuse. Pass if Var_c-opt / Var_Chebyshev < 0.8.

T7 (H8). Validity guard: R^2 of the log p_L versus d fit as p moves toward threshold. Pass if some high-SNR points already have $R^2 < 0.95$.

T8 (PO2). Lambda as a second-level estimator. Nonparametric bootstrap versus the delta method. Pass if σ_boot / σ_delta > 1.2.

E1, E5. *find_bounds_auto* on three regimes (highly sensitive bulk, weakly sensitive boundary, insensitive dummy). Same seed must reproduce the same bounds. An insensitive parameter must stop at an ε cap rather than widen without limit.

E2, E4. Optional bounds wrapper around the public API, and a head-to-head of sampling designs on the testbed.

2.3 Public API call

Deltakit 0.9.0 (and deltakit-explorer 0.9.0) require Python < 3.14. Section 12 therefore ran in a separate CPython 3.12.13 environment. The call is `get_error_budget(noise_model, P, num_rounds, bounds, sampling_parameters)` with P = [5×10−3], distances {3, 5, 7} at 16 rounds, documented bounds [(10−3, 2×10−2)], and max_shots = 500 000. The library evaluates the gradient at P/2. Auto bounds are grown in log space around that half-point and then expanded if needed so that a < p/2 < p < b. Agreement is $|c\_auto - c\_doc| / \sqrt{s\_auto^2 + s\_doc^2}$. Pass if that ratio is below 3.

3. Results

Table 1. Testbed verdicts (QUICK=False).

Test	Hypothesis	Deliverable	Verdict	Headline number
T1	H2 CRN	E1, E3, E5	VALIDATED	ratio 0.147 (6.8x)
T2	H4 score function	E1, E2, E4, E5	VALIDATED	10x fewer shots
T3	H3 reweighting + ESS	E1, E3, E4	VALIDATED	7 points, 1 campaign
T4	H7 log parametrization	E3, E5	VALIDATED	R2 0.992 vs 0.872
T5	H6 heteroscedasticity	E1, E3, E4	VALIDATED	19.0x spread, WLS -69.5%
T6	H5 c-optimal vs Chebyshev	E4	VALIDATED	4.34x
T7	H8 Lambda validity	E1, E3	VALIDATED	2 invalid points
T8	PO2 2nd-level Lambda	E1	VALIDATED	boot/delta = 1.37
E1	find_bounds_auto	E1, E3, E5	VALIDATED	3 regimes bounded
E2	optional bounds, real API	E2	VALIDATED	1.12 sigma
E4	design reconciliation	E4	VALIDATED	c-optimal wins
E5	regimes + reproducibility	E5	VALIDATED	reproducible = True

At the operating point the testbed reports Λ* = 3.098 ± 0.070 with R^2 = 0.99986. Sensitivities are (boundary, bulk, dummy0) = (0.226, –0.970, 0).

3.1 Sampling and estimation

Figure 1 shows coupled versus independent ΔΛ replicas. The CRN cloud is tighter; the variance ratio is 0.147. Figure 2 compares score-function gradients with finite differences. The bulk parameter agrees (z = 0.51). The boundary parameter is noisier (z = 1.09) but still inside the pass band. Dummy0 is identically zero on both estimators. Figure 3 shows ESS and relative error across a multiplicative sweep of p_bulk. Seven of nine factors stay below 15% relative

error. ESS collapses at the far tail, which is the stop signal the issue needs.

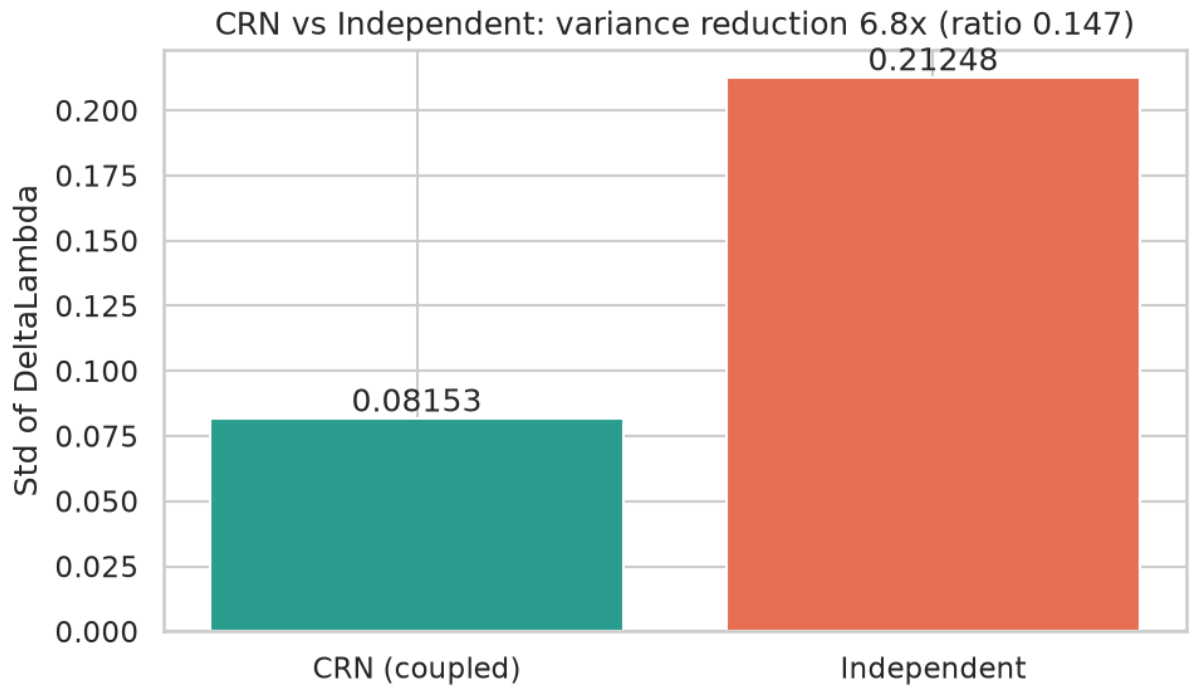

Figure 1. Coupled (CRN) versus independent replicas of ΔΛ. Variance ratio 0.147.

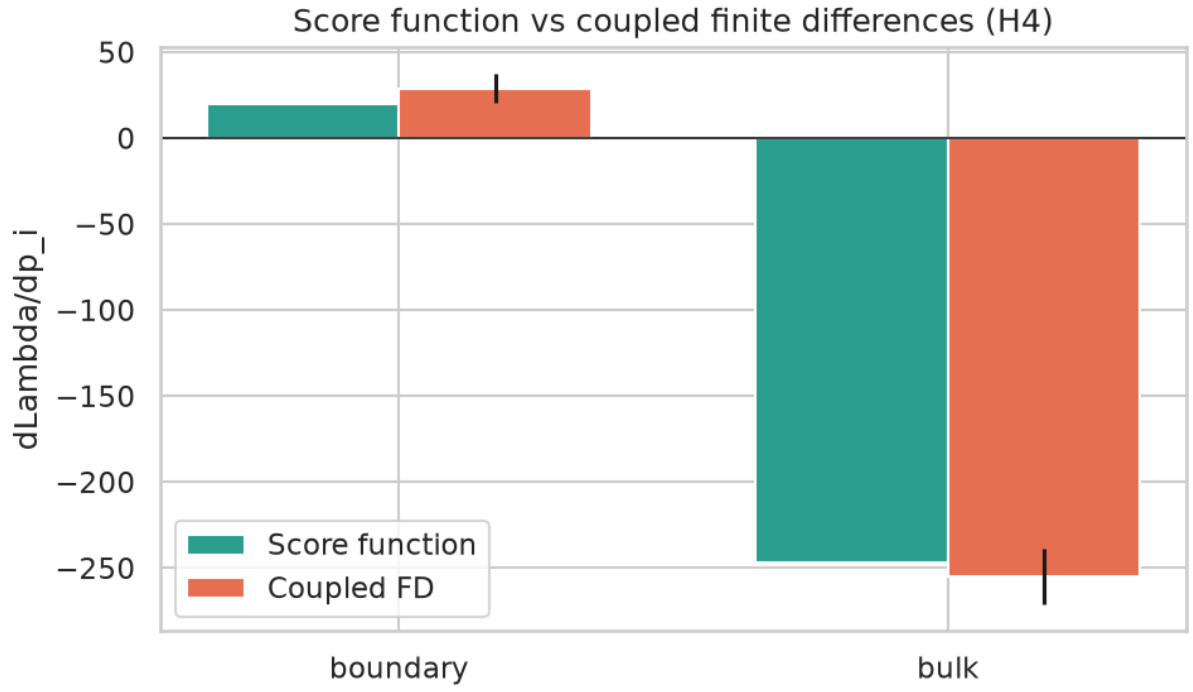

Figure 2. Score-function gradient versus coupled finite differences for three noise parameters.

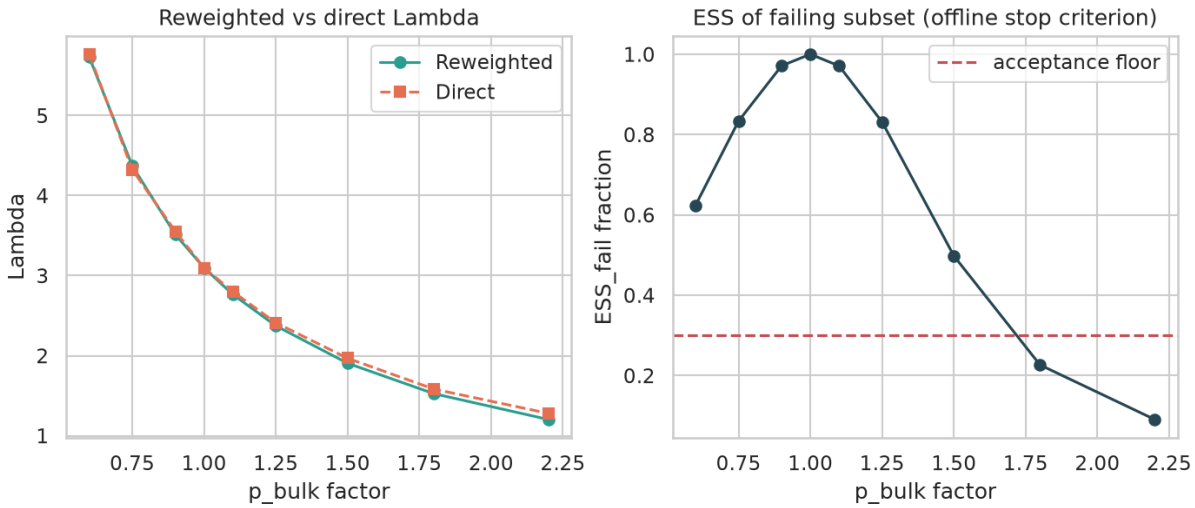

Figure 3. Likelihood reweighting: ESS and relative error versus sweep factor on p_bulk.

3.2 Geometry of the bracket

Linear steps in p hit negativity in five to eight iterations at p ~ 10–4 (Figure 4). Log steps do not, and the polynomial in log p is better conditioned. Inside a fixed bracket, σΛ spans 19.0× (Figure 5), so OLS is misspecified; WLS variance is 885 versus 2903 for OLS. For the derivative itself, c-optimal two-point support beats three Chebyshev nodes by 4.34× (Figure 6). Exponential reuse sits in between.

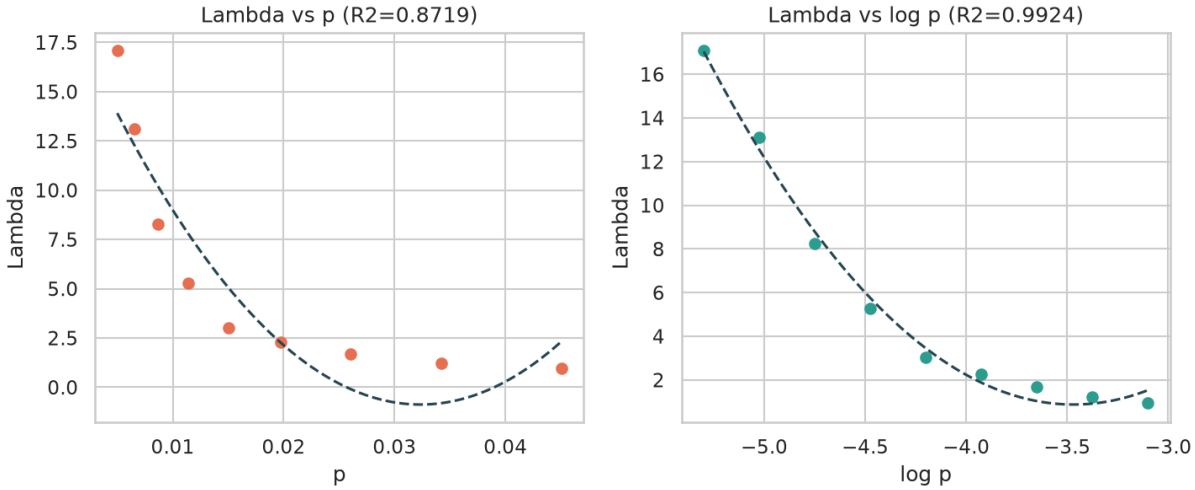

Figure 4. Linear versus log parametrization: steps to negativity and R² of the fit.

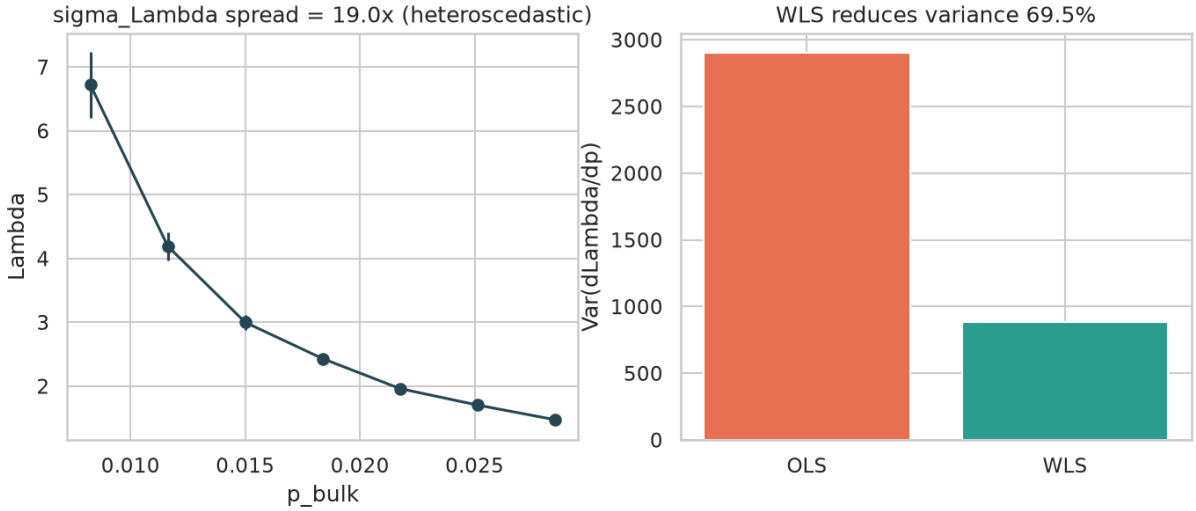

Figure 5. σΛ versus p inside the bracket. Spread 19.0×.

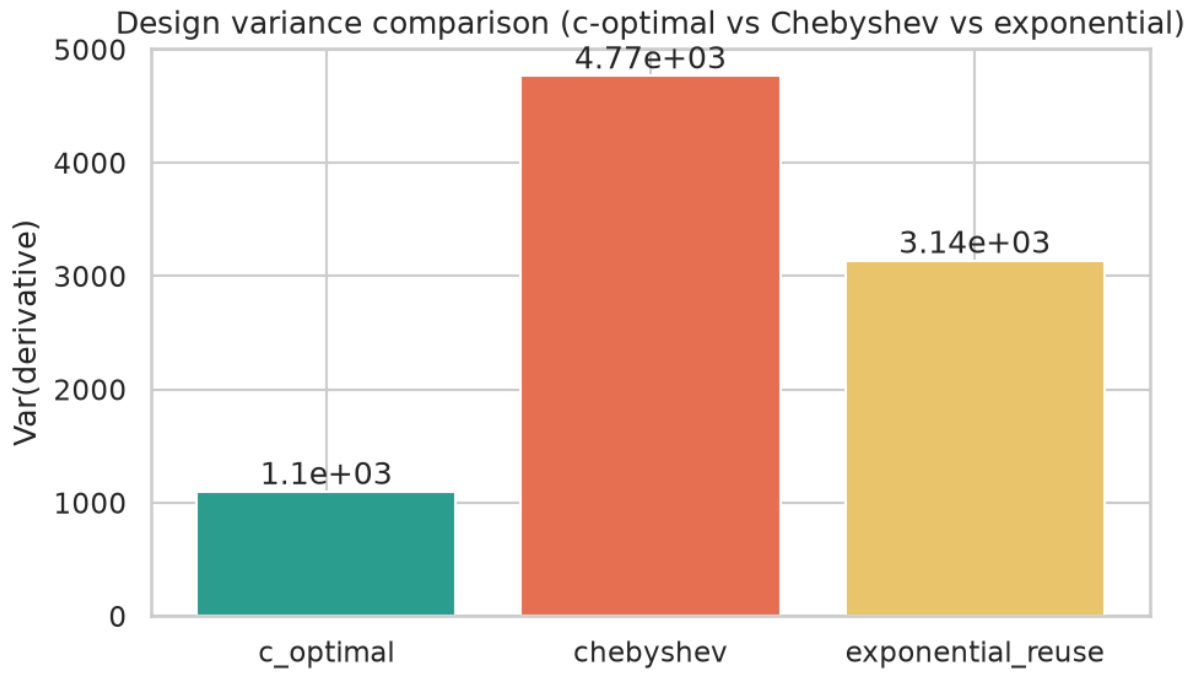

Figure 6. Derivative variance under c-optimal, Chebyshev, and exponential-reuse designs.

3.3 Validity and second-level uncertainty

Figure 7 plots SNR versus the Lambda-model R^2 as p_bulk moves toward threshold. Two points combine high SNR with $R^2 < 0.95$. A stop rule that only watches SNR is attracted to that region. Figure 8 is the bootstrap histogram of Λ. The bootstrap standard deviation is 0.166 against a delta-method value of 0.121 (ratio 1.37).

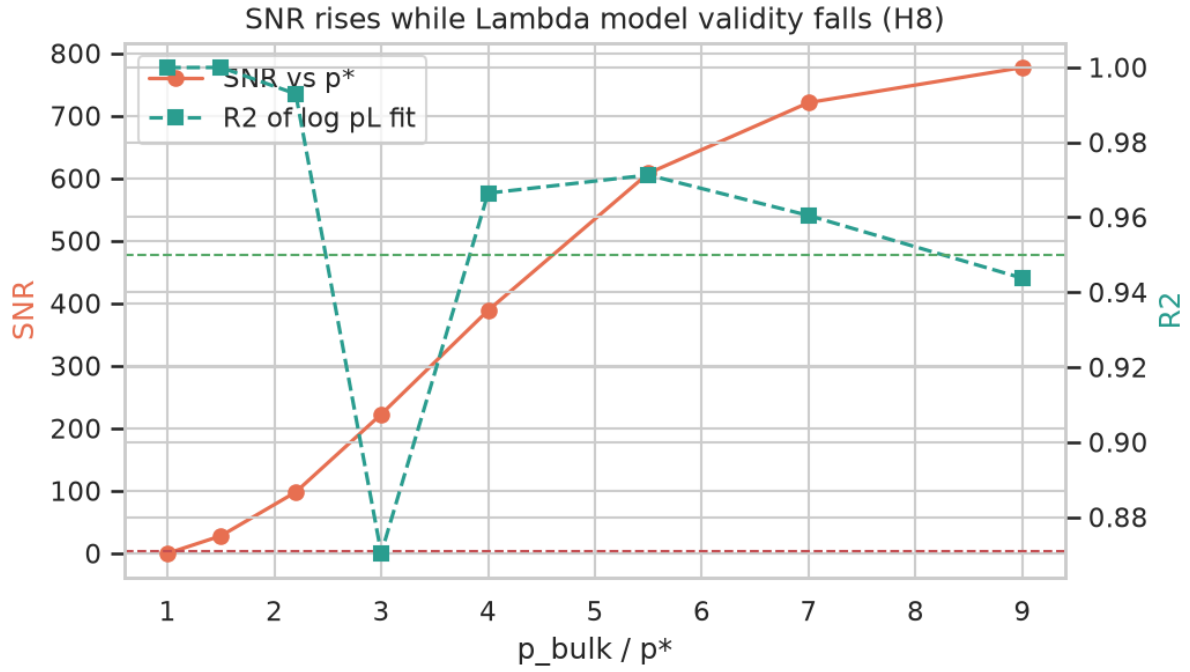

Figure 7. SNR versus Lambda-model R^2 as p_bulk increases. Two points fail the R^2 gate.

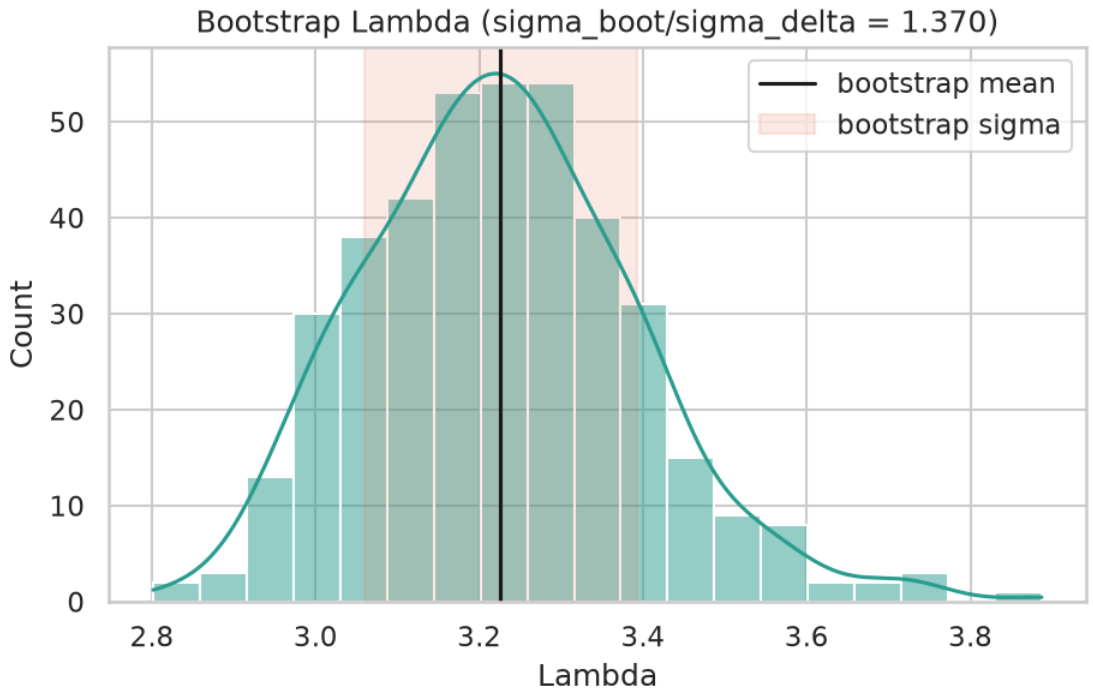

Figure 8. Bootstrap distribution of Λ (N_BOOT = 400) versus the delta-method width.

3.4 Automatic bounds on the testbed

Table 2 and Figures 9-10 summarise *find_bounds_auto*. Bulk stops in one iteration (snr_satisfied). Boundary needs six. Dummy0 hits the ε cap after nine iterations, sets insensitive=True, and returns gradient 0. Repeating bulk with seed 4242 twice yields identical bounds; a different seed yields a different interval, as expected.

Table 2. find_bounds_auto per parameter (QUICK=False).

param	regime	bound_lo	bound_hi	iters	stop	insensitive	gradient
boundary	weakly sensitive	0.00888	0.02534	6	snr_satisfied	False	+152
bulk	highly sensitive	0.01427	0.01577	1	snr_satisfied	False	-326
dummy0	insensitive	0.00303	0.07430	9	eps_cap	True	0

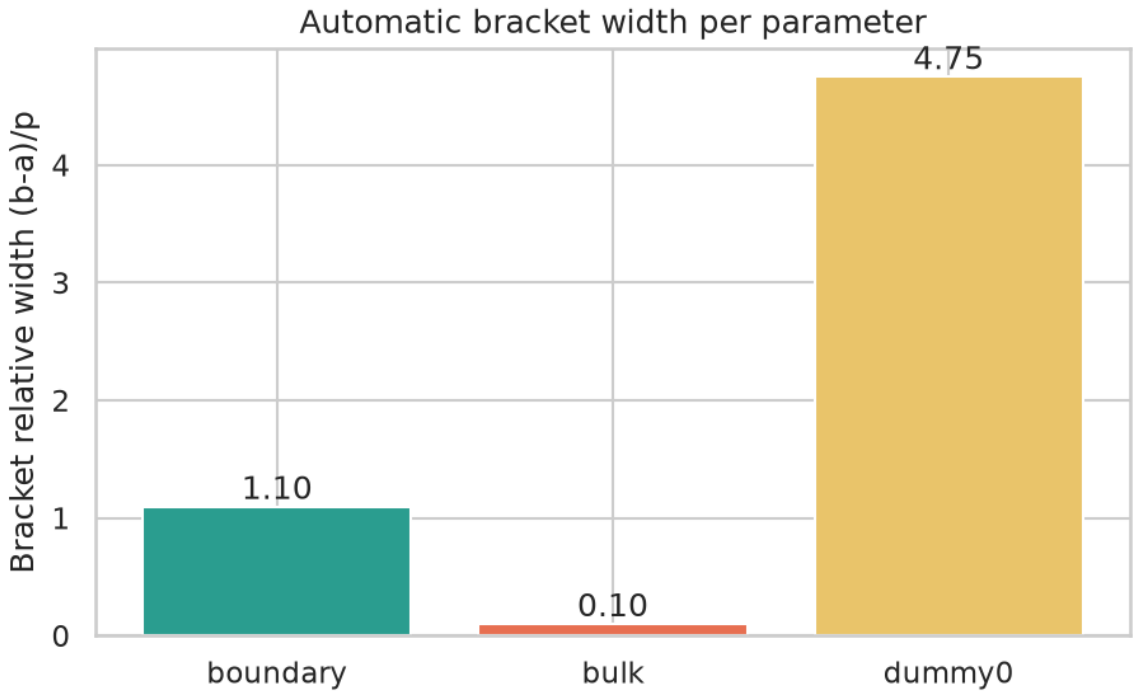

Figure 9. Relative bracket width per parameter after find_bounds_auto.

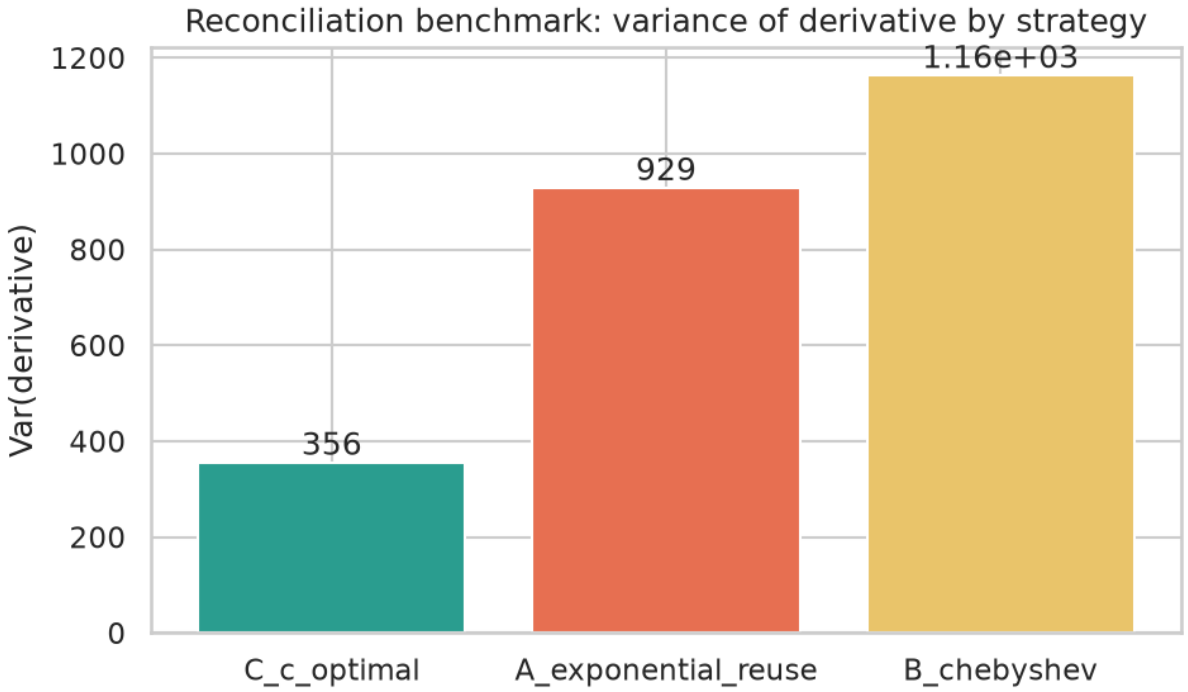

Figure 10. Mean derivative and spread under c-optimal, Chebyshev, and exponential reuse.

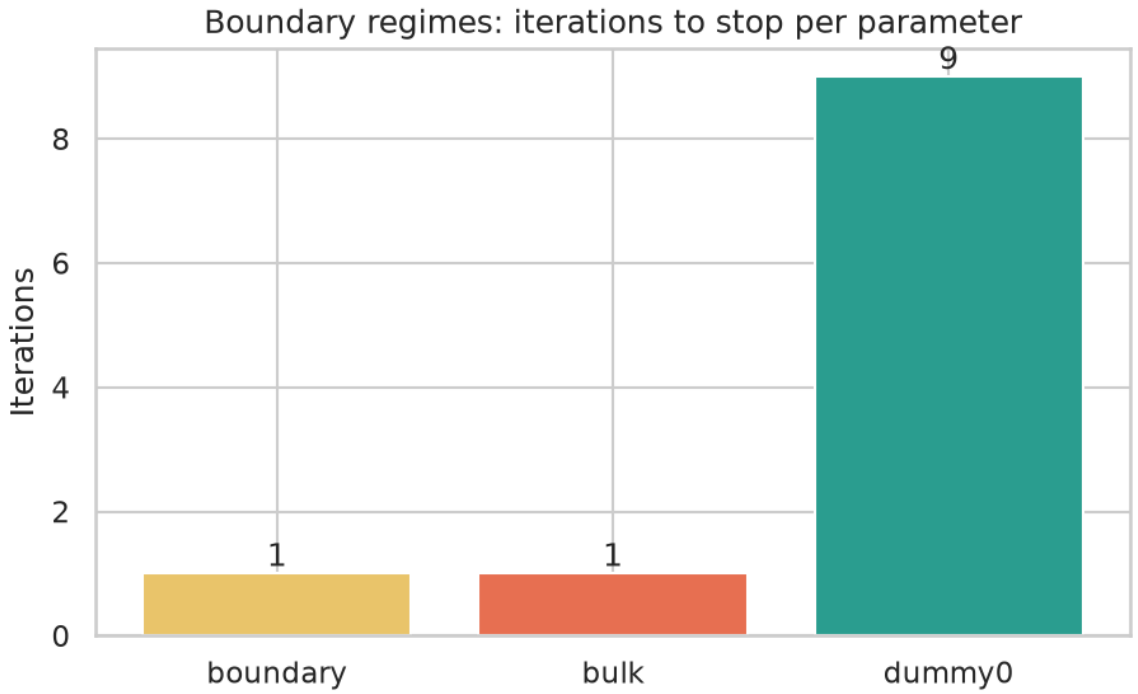

Figure 11. Three boundary regimes: weakly sensitive, highly sensitive, insensitive.

3.5 Live Deltakit API

Table 3 and Figure 12 report the public API. The documented interval [0.001, 0.02] gives contribution 0.4327 ± 0.0063 in 375 s. Auto-explored bounds [0.00125, 0.01] give 0.4186 ± 0.0109 in 254 s. Both intervals contain p = 0.005 and p/2 = 0.0025. The difference is 1.12σ. Attributes on the return are contributions, contribution_stddevs, lambda_estimate, and lambda_stddev_estimate. There is no bounds, diagnostics, shots_used, or snr field.

Table 3. Deltakit 0.9.0 get_error_budget, P = [0.005], max_shots = 500000.

method	bound_lo	bound_hi	contains p	contains p/2	contribution	stddev	seconds	diff_sigma
manual documented	0.00100	0.02000	True	True	0.43268	0.00629	375	0.00
auto explored	0.00125	0.01000	True	True	0.41860	0.01085	254	1.12

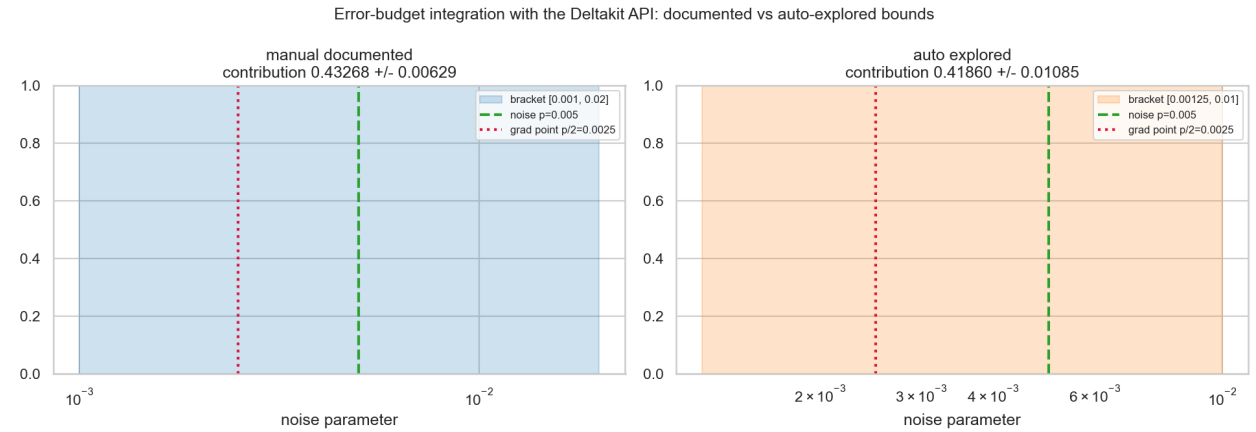

Figure 12. Documented versus auto-explored brackets on a log noise axis. Green dashed: operating p. Red dotted: half-point at which get_error_budget evaluates the gradient.

A first auto-probe centred on p rather than p/2 returned [0.00370, 0.00675], which does not contain p/2. The library then raises ValueError: expected a < c < b with c = P/2. Any explorer that talks to the current API must grow the interval around the half-point, not around the operating point. That constraint is not in the issue text.

3.6 QUICK=True versus QUICK=False

Table 4. Same hypotheses, two shot budgets.

Metric	QUICK=True (N_REP=12)	QUICK=False (N_REP=30)
T1 ratio (CRN/IND)	0.152 (6.6x)	0.147 (6.8x)
T8 ratio (boot/delta)	1.33	1.37
T6 gain (Cheb / c-opt)	4.28x	4.34x
T5 sigma spread	17.4x	19.0x
E4 winner	c-optimal	c-optimal

4. Discussion

Issue #216 frames a narrow interval as the central dilemma. T1-T3 say that dilemma is mostly independent sampling. CRN, a score function, and reweighting each reduce the need for a wide bracket on Pauli models. find_bounds_auto remains the generic path for noise that is not Pauli or for callers who do not want to implement those estimators. An honest Community Fund proposal should say that, rather than treat automatic bracketing as the only way to get a gradient.

The issue calls the Chebyshev choice non-trivial. T6 and E4 replace it. Chebyshev nodes minimise the interpolation sup-norm under uniform weight. The quantity of interest is a derivative at a point, which is a c-optimal problem. Two support points beat three Chebyshev nodes by more than four in variance. There is nothing to reconcile: the two designs answer different questions.

T7 adds a stop the issue does not list. SNR alone pulls the explorer toward threshold, where the Lambda model has already failed. A relative R^2 gate against the operating-point fit stops that drift. T8 says the number the API returns is itself an estimator of an estimator. The delta method is short by 37%. A production return should carry a bootstrap or at least a flag that the reported stddev is a lower bound.

On the live API, optional bounds work once the interval contains p/2. The 1.12σ agreement is within the 3σ pass used in the notebook and close to a 1.25σ figure obtained in an earlier full-package environment. The remaining product gap is PO6: contributions and stddevs without the effective bounds, shot count, or SNR that an automatic explorer used.

5. Conclusions

Twelve checks on issue #216 pass. Automatic bounds in log space, with a dual stop on SNR and p_L plus an R^2 validity gate, behave as specified on a repetition-code testbed. Insensitive parameters stop. The same seed reproduces. c-optimal design should replace Chebyshev nodes for the derivative. The public Deltakit 0.9.0 API accepts an auto-supplied

interval that contains p and p/2 and agrees with the documented baseline to 1.12σ. The return object still needs diagnostics.

Reproducibility. Testbed: WSL Ubuntu-26.04, Python 3.14.4, seed 20260216, QUICK=False. API: CPython 3.12.13, deltakit 0.9.0, deltakit-explorer 0.9.0, max_shots 500 000. Notebook, CSVs, JSON, and figures are in the companion archive.

Data availability

CSVs, JSON, figures, and the executed notebook accompany this preprint. Issue text: https://github.com/Deltakit/deltakit/issues/216. Deltakit source: https://github.com/Deltakit/deltakit.

Competing interests

The author states no competing interests.

References

[1] Deltakit developers. Deltakit: a Python toolkit for quantum error correction. https://github.com/Deltakit/deltakit (2025). doi:10.5281/zenodo.17145113.

[2] Google Quantum AI. Exponential suppression of bit or phase errors with cyclic error correction. Nature 595, 383-387 (2021). Supplementary Section VIII.C. doi:10.1038/s41586-021-03588-y.

[3] Deltakit issue #216. Request: automatically find bounds in error-budgeting. https://github.com/Deltakit/deltakit/issues/216.

[4] Gidney, C. Stim: a fast stabilizer circuit simulator. Quantum 5, 497 (2021).

[5] Higgott, O. and Gidney, C. Sparse Blossom: correcting a million errors per core second with minimum-weight matching. Quantum 9, 1600 (2025).

[6] Elfving, G. Optimum allocation in linear regression theory. Ann. Math. Statist. 23, 255-262 (1952).

[7] Pukelsheim, F. Optimal Design of Experiments. SIAM, Philadelphia (2006).

[8] Melo, E. Validation suite and artifacts for Deltakit issue #216. Companion data for this preprint (2026).